Contents

Volume 03 — VAE family (L21–L30) **1**
L21: Introduction to VAE and the working of the encoder 1
L22: Probabilistic decoder, ELBO, and the VAE loss 2
L23: Reparameterization trick 3
L24: VAE numerical example 4
L25: Practical exercise 1 — variational autoencoder 5
L26: Entanglement, disentanglement, and β-VAE 6
L27: Conditional VAE 7
L28: Latent-space interpolation 7
L29: Practical exercise 2 — Conditional VAE 8
L30: Practical exercise 3 — β-VAE 9

Volume 03 — VAE family (L21–L30)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L21: Introduction to VAE and the working of the encoder

Learning focus

- Contrast autoencoders (**reconstruction**) with VAEs (**generation** of new samples).
- State the generative goal: from finite unlabeled samples, learn $p_\theta(x) \approx p_{\text{data}}(x)$.
- Explain why the encoder learns an **approximate posterior** $q_\phi(z \mid x)$, not the true $p(z \mid x)$.
- List the **Gaussian** assumptions on $p(z)$ and $q_\phi(z \mid x)$, and what the encoder actually outputs (**mean** and **log-variance**).
- Walk the lecture's **latent-size-5** sampling picture.

Exam lines

- AE reconstructs; VAE must **generate** by learning $p_\theta(x)$ from unlabeled samples.
- Encoder's target is $p(z \mid x)$, but evidence $p(x) = \int p(x \mid z)p(z)\,dz$ is **intractable**, so we train $q_\phi(z \mid x)$.
- Prior $p(z)$ and approximate posterior q_ϕ are both taken **Gaussian**; encoder emits μ **and** $\log \sigma^2$; **KL** pins q to the prior.
- z is a **vector**; sampling it is a **random pick** from each coordinate's Gaussian, so one x has **infinitely many** latents.

Concept tables

Object	Distribution	Lives in
Data $x_1, \dots, x_n$	$p_{\text{data}}(x)$	**Data space** (input dimension / structure)
Latent z	$p(z)$	**Latent space** (different dimension / structure)
Autoencoder	VAE	
———--	—–	
Encoder	**Probabilistic encoder**	
Latent code	Latent **distribution**	
Decoder	**Probabilistic decoder**	

L22: Probabilistic decoder, ELBO, and the VAE loss

Learning focus

- Describe the decoder as a net that maps sampled z to **likelihood parameters** (Gaussian vs Bernoulli, by data type).
- Use the lecture's **5-feature / latent-size-2** example to see **infinitely many** z for one x, and slightly different $\hat{x}$ that stay near x.
- Explain why $p(x) = \int p(x \mid z)p(z)\,dz$ is **intractable**, so the encoder never learns the **true** posterior.
- Write **ELBO** as reconstruction minus KL, and the **VAE loss** as **minimize** $-$ELBO.
- State the **closed-form Gaussian KL** and why **naïve sampling of** z **blocks backprop** (reparameterization is the next video).

Exam lines

- Decoder outputs **likelihood parameters**: Gaussian (μ, σ^2) for continuous x, Bernoulli probabilities for binary x.
- One $x \mapsto$ **infinitely many** $z \mapsto$ many nearby $\hat{x}$ (**generation**).
- $p(x) = \int p(x \mid z)p(z)\,dz$ sums **unbounded** contributions; true posterior is **intractable**; encoder uses q_ϕ.
- Maximize **ELBO** $= \mathbb{E}[\log p(x \mid z)] - \mathrm{KL}(q_\phi \,\|\, p)$. Minimize **loss** $= -$ELBO **=** reconstruction (MSE or BCE) + KL.
- Naïve sampling of z **blocks** backprop → reparameterization next.

Concept tables

Data you reconstruct / generate	Likelihood	Decoder outputs	How you get $\hat{x}$
Continuous / real values	**Gaussian**	**Mean and variance**	Theory: **sample** $\hat{x}$ from that Gaussian. Practice (next numerical lecture): the **mean itself** is $\hat{x}$
Binary (each feature / pixel 0 or 1)	**Bernoulli**	A **probability per feature**	$\hat{x}$ sampled from those Bernoullis

Sample	(z_1, z_2)
$z^{(1)}$	$(0.56, -0.11)$
$z^{(2)}$	$(0.43, -0.32)$
$z^{(3)}$	$(0.61, -0.24)$

L23: Reparameterization trick

Learning focus

- Restate the VAE loss (reconstruction + closed-form KL) and why **sampling** z blocks **backprop**.
- Show that **decoder** parameters are fine under ordinary chain rule; the break is $\partial z/\partial \mu$ and $\partial z/\partial \sigma$.
- Write the **reparameterization** $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, and treat ε as a **constant** during backprop.
- Compute $\partial z/\partial \mu = 1$ and $\partial z/\partial \sigma = \varepsilon$, then push gradients into the **encoder weights**.
- Draw the **full VAE architecture** the course withheld until this trick.

Exam lines

- Reconstruction + KL is not enough: **sampling** z is non-differentiable, so encoder ϕ cannot be updated.
- Decoder W, b and even $\partial L/\partial z$ are ordinary calculus; the hole is $\partial z/\partial \mu$ and $\partial z/\partial \sigma$.
- Reparameterization: $z = \mu + \sigma\varepsilon$ with ε **independent** and treated as **constant**. Then $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$.
- Gradients reach encoder parameters; **end-to-end** VAE training works.

Concept tables

Data	Output activation f
Binary	**Sigmoid**
Continuous	**Linear**

Data	Output activation f
Before reparameterization	After
————————	——–
$z \sim \mathcal{N}(\mu, \sigma^2)$ (raw sample)	$z = \mu + \sigma \odot \varepsilon, \varepsilon \sim \mathcal{N}(0, I)$

L24: VAE numerical example

Learning focus

- Run a **full forward pass** of a tiny MLP VAE on one **continuous tabular** row (four features).
- Compute hidden ReLUs, **μ** and **log-variance** (no activation on those heads), **σ²** and **σ**, then **reparameterized** z.
- Decode with ReLU then **linear** output; compare $\hat{x}$ to x.
- Plug the lecture's numbers into **MSE** reconstruction and **closed-form Gaussian KL**, then add them.

Exam lines

- Tiny MLP VAE on $x = (5.1,\ 3.5,\ 1.4,\ 0.2)$: hidden ReLU **(1.07, 1.93, 0.87)** → **μ=(0.043, 0.551)**, **log σ²=(0.732, 0.302)**, **σ₁=1.442**.
- First $\varepsilon = (0.5, -0.2)$ → $z = (0.764,\ 0.318)$; other ε give other z from the **same** Gaussian.
- Decoder ReLU **(0.6538, 0, 0.0938)** → linear $\hat{x}_1 = 0.10262$; reconstruction is bad because weights are random.
- $L_{\text{recon}} = 9.7111$, KL $= 0.352$, $L = 10.0631$. Training would now update parameters. Week 4 lab implements a real VAE.

Concept tables

Choice	Lecture
Data	**Continuous**, **tabular** (rows × columns)
Features / input neurons	**4**
Spoken coordinates	**5.1**, **3.5**, **1.4**, and a fourth input into weight **0.2**
Consistent fourth value	**0.2** (Iris-style row $(5.1,\ 3.5,\ 1.4,\ 0.2)$ makes the spoken $h_1 = 1.07$)
Encoder hidden	**3** neurons, **ReLU**, bias **0.1**
Latent size	**2** (two μ, two log-σ²)
Decoder hidden	**3** neurons, **ReLU**, bias **0.1**
Decoder output	**4** neurons, **linear** (continuous reconstruction)

Choice	Lecture
Weights	**Random** (can be negative, e.g. -0.7)
Hidden unit	Pre-activation h
———	——————-
1	**1.07**
2	**1.93**
3	**0.87**

L25: Practical exercise 1 — variational autoencoder

Learning focus

- Contrast a **vanilla autoencoder** (fixed latent vector) with a **VAE** ($z = \mu + \sigma\varepsilon$).
- Build the lecture's Keras VAE on **MNIST**: **784 → 256 → 16** latents → **256 → 784**, with a **sampling layer**.
- Code **BCE reconstruction + closed-form KL**, train with **Adam**, **10** epochs, batch **128**.
- Plot original vs reconstructed digits and state the lecture's diagnosis: missing **labels** → blurry / wrong digits, which a **conditional VAE** will fix.

Exam lines

- VAE lab: MNIST **60k / 10k**, $/255$, flatten **784**, **no labels**.
- Encoder **256 ReLU + 16-D** μ and log-σ²; $z = \mu + \exp(0.5 \log \sigma^2) \odot \varepsilon$; decoder **256 ReLU → 784 sigmoid**.
- $L = \text{BCE} + \text{KL}$ with the Gaussian closed-form KL; **Adam**, **10** epochs, batch **128**.
- Final losses ≈ KL 3.4×10^{-5}, recon **0.2643**, total **0.2644**; reconstructions are **784-D** again.
- Plots show **class mix-ups** because y is unused — motivation for **CVAE**.

Concept tables

Autoencoder	VAE in this lab
Encoder x → **one fixed** latent vector	x → **μ and variance** of the pixel distribution
Latent Cannot be resampled	$z = \mu + \sigma\,\varepsilon$ with noise **ε**
Decoder z → reconstruction	Same, but z is **stochastic**

Import	Use
`tensorflow as tf`	Deep-learning framework
`keras.layers`	Layers: input, hidden / output layers;
`keras.models`	Models

Autoencoder	VAE in this lab
`matplotlib.pyplot as plt`	Original vs reconstruction plots

L26: Entanglement, disentanglement, and β-VAE

Learning focus

- Explain why a fully connected encoder produces an **entangled** z: each coordinate mixes many factors of variation.
- Contrast that with a **disentangled** latent space, where each z_i mainly controls one factor (hair, lighting, smile, …).
- Write the **β-VAE** loss and say what $\beta = 1$ vs $\beta > 1$ does to the KL term.
- Use the lecture's two-dimensional KL examples to connect "small μ" ↔ closer to the prior ↔ one-factor codes.
- List the three practical fixes when there are **more factors than latent dimensions**: larger z, moderate β, **β-annealing**.

Exam lines

- Fully connected μ, σ make each z_i a **mixture** of factors: that is **entanglement**. Editing one coordinate moves several attributes.
- **Disentanglement** means one coordinate ≈ one factor (hair vs smile vs pose on the face grids; thickness vs slant on digits).
- β-VAE is the VAE loss with weight β on the KL. $\beta = 1$ is ordinary VAE; $\beta > 1$ **encourages** (does not certify) disentanglement; β too large **hurts reconstruction**.
- Small μ ⇒ small KL ⇒ codes closer to the prior and less mixed; large μ ⇒ large KL ⇒ mixed, high-capacity z.
- If factors outnumber latent dimensions, raise **latent size**, use a **moderate** β, or **anneal** β from ≈ 0 upward.

Concept tables

Factor	What it means on a digit
Thickness	Thin vs thick stroke
Slant	Tilted vs upright
Rotation	How the glyph is rotated
Position	Where the digit sits in the frame

Coordinate	Should mainly control
z_1	Hair
z_2	Lighting
z_3	Skin tone
z_4	Smile

Factor	What it means on a digit
z_5	Pose

L27: Conditional VAE

Learning focus

- Recap the **unconditional** VAE path $x \to z \to \hat{x}$ and why generation is not class-controlled.
- State the CVAE change: extra condition y into **both** encoder and decoder.
- Explain why z in a CVAE can drop class identity and keep **within-class style**.
- Write the CVAE loss as the VAE loss with y inserted, and follow the lecture's numerical comparison (4 features $\to$ 6 inputs with one-hot y).

Exam lines

- Vanilla VAE: $x \to z \to \hat{x}$. You cannot request a **specific class**.
- CVAE: pass y **into encoder and decoder**. Reparameterization becomes $z = \mu(x, y) + \sigma(x, y) \odot \varepsilon$.
- z no longer has to store class identity, so it can store **within-class style**; generation is **controlled** by y.
- Loss is the VAE pair (reconstruction + KL) with y in both q and p.
- In the reused week-3 example, four features plus a 2-d one-hot become **six** inputs, extra weights, a **richer** hidden code, and a **different** $\hat{x}$.

Concept tables

Network	Standard VAE input	CVAE input
Encoder	x	x **and** y
Reparameterization	$\mu(x), \sigma(x)$	$\mu(x, y), \sigma(x, y)$
Decoder	z	z **and** y
Object	Dimension	
——–	———–	
x	4	
one-hot y	2	
encoder input $[x; y]$	**6**	

L28: Latent-space interpolation

Learning focus

- State why interpolation is done in z, not in pixel space.

- Write the linear blend $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ and evaluate the lecture's $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$.
- Decode each intermediate z to get a **smooth morph** (digit 3 → digit 8).
- Name the medical **missing-scan** use case (early tumor vs late tumor, nothing in between).

Exam lines

- Interpolation for generation happens in **latent space**, then you **decode**. You cannot mix the two photographs directly and call it a VAE interpolation.
- $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ with $\alpha \in [0, 1]$. Endpoints recover the two encodings; $0.25/0.5/0.75$ are the lecture's interior steps.
- Componentwise mix if $\dim z > 1$. Each mixed z has the **same** length as z_A and z_B.
- Visual result: a **systematic** morph (3 → 3-like → mix → 8-like → 8), useful when **intermediate observations are missing** (tumor-progression example).

Concept tables

α	Algebra	Whose code you get
0	$1 \cdot z_A + 0 \cdot z_B$	exactly z_A → decode to image **3**
1	$0 \cdot z_A + 1 \cdot z_B$	exactly z_B → decode to image **8**
in between	a convex combination	a code **on the segment** $z_A \to z_B$
Code you decode	Image you should see	
———	——————-	
$z_A = z(0)$	$\hat{x}_A$ (reconstructed 3)	
$z(0.25)$	3-like intermediate	
$z(0.50)$	mixed 3/8	
$z(0.75)$	8-like intermediate	
$z_B = z(1)$	$\hat{x}_B$ (reconstructed 8)	

L29: Practical exercise 2 — Conditional VAE

Learning focus

- Build a **CVAE** on **MNIST** in TensorFlow/Keras, using the **same** 28×28 digits as the previous VAE lab so the two are comparable.
- One-hot the labels, concatenate y to **encoder and decoder**, keep the VAE sampling layer.
- Train with **BCE reconstruction + KL**, **Adam**, and generate a **class-specific** digit (the demo uses **7**).

Exam lines

- CVAE = VAE sampling + **one-hot** y **concatenated on both sides**.
- MNIST, 784, latent **16**, hidden **256 ReLU**, output **sigmoid 784**, **Adam**, **10** epochs, batch **128/256**.
- Loss is still **BCE + KL**; epoch-10 total 0.2160, smaller than the unconditioned VAE run.
- Decode $(z, y{=}7)$ to emit **that** class on demand.

Concept tables

Import	Role in this notebook
TensorFlow	Deep-learning framework
Keras `layers, Model`	Input / hidden / output layers; wrap encoder, decoder, CVAE
NumPy	Arrays; one-hot is numeric
`matplotlib.pyplot` as `plt`	Show the generated digit

Digit	One-hot (length 10)
—:	———————
0	`1 0 0 0 0 0 0 0 0 0`
1	`0 1 0 0 0 0 0 0 0 0`
2	`0 0 1 0 0 …`
9	`0 0 0 0 0 0 0 0 0 1`

L30: Practical exercise 3 — β-VAE

Learning focus

- Implement **β-VAE** on MNIST: same encoder/decoder skeleton as the VAE labs, with a scalar β **multiplying the KL**.
- Sweep $\beta \in \{0.5, 1, 2, 4, 10\}$ and record reconstruction vs KL.
- Read the plot **"effect of beta on beta variational autoencoder"** and the reconstructed **digit 7** as β grows.

Exam lines

- β-VAE loss: RL $+ \beta \cdot$ KL. $\beta = 1$ is vanilla VAE; sweep **0.5, 1, 2, 4, 10**.
- MNIST **60k / 10k × 784**, latent **10**, **256 ReLU**, **sigmoid 784**, **Adam**, **5** epochs, batch **128**, **no** labels.
- Measured trade-off: $\beta \uparrow \Rightarrow$ **RL↑, KL↓**, digit **7** turns to mush by $\beta = 10$.
- Plot title and markers as above; use the trackers so all five runs are comparable.

Concept tables

Split	Samples	Features
Train	**60,000**	784
Test	**10,000**	784
Name	Value in the demo	
——	————————-	
Latent dimension	**10** (784 down to 10 codes). The sampling-layer recap still says “16” once — leftover wording from the VAE lab; the **decoder input and the run** use **10**.	
β grid	`[0.5, 1, 2, 4, 10]`	
Epochs	**5** (kept small because **five models** train; you may raise it)	
Batch size	**128**	
Optimizer	**Adam**	
`verbose`	**1** (one-line loss print per epoch)	
